YT Archive

VERSION 1.0

Technical Specification, Architecture & Operational Manual

YT Archive is a desktop application written in Python (CustomTkinter) engineered for high-performance YouTube channel archiving, metadata harvesting, and stream extraction. Leveraging `yt-dlp` and `FFmpeg`, it facilitates non-transcoded multiplexing of discrete video and audio tracks, preserving source bitrate and master fidelity.

Core Architecture & System Stack

UI & Thread Orchestration

Built on **CustomTkinter** with asynchronous UI scheduling via `root.after()`. Channel scanning runs on background daemon worker threads to prevent main-thread latency during heavy JSON deserialization.

Stream Extraction Engine

Powered by **yt-dlp**. Default profiles invoke composite stream queries (`bestvideo+bestaudio`). Formats are strictly sorted by resolution, framerate, color depth (HDR), codec efficiency, and container.

Multiplexing & Merging

Uses **FFmpeg** for stream multiplexing into `.mkv` without video transcoding. Includes an automatic binary fallback through `imageio-ffmpeg` if a system-level binary is missing from PATH.

Atomic Data Persistence

Exports normalized metadata to `.metadata.json` using an atomic temp-file swap (`os.replace`). Channel exports write standard BOM-encoded UTF-8 CSVs for spreadsheet compatibility.

Quality Resolution Matrix

Target Mode	yt-dlp Selector Syntax	Container	Transcode Cost
Best Available	`bestvideo+bestaudio/bestvideo/best`	MKV	None (Stream Copy)
2160p (4K UHD)	`bestvideo[height<=2160]+bestaudio/...`	MKV	None (Stream Copy)
1080p (FHD)	`bestvideo[height<=1080]+bestaudio/...`	MKV	None (Stream Copy)
720p / 480p / 360p	`bestvideo[height<=N]+bestaudio/...`	MKV / MP4	None (Stream Copy)
Audio Only	`bestaudio/best` (post-proc: libmp3lame)	MP3	Lossy (320 kbps CBR)

Operational Boundary & Quality Note

Stream Fidelity Notice: YouTube encodes all incoming uploads through proprietary VP9, AV01, or H.264 pipelines. YT Archive extracts the maximum-fidelity distribution stream available. It preserves complete integrity without secondary compression, but cannot restore data pruned by YouTube's ingest algorithms.

Environment Setup & Deployment

YT Archive runs on 64-bit Windows environments running Python 3.10 or higher. Follow the virtual environment provisioning instructions below:

```
# Open terminal in root directory and initialize isolated environment python -m venv .venv .venv
```

Standard Operating Workflow

1. Channel Metadata Extraction

Paste a valid YouTube channel URL (e.g. `https://www.youtube.com/@ChannelHandle`). Click **Fetch All Uploads**. The application extracts channel descriptors and initiates a streaming parse of the video catalog, displaying elapsed runtime, scanned items, and transfer rate.

2. Card Interaction, Filtering & Search

The channel grid populates up to **48 video cards per page**. Use the navigation buttons to advance across pages. The title filter implements debounced input monitoring: typing will filter across the entire cached catalog without freezing UI responsiveness.

3. Batch Queue & Single Download Management

To archive individual assets, select **Download** on any card, choose the target resolution, and click **Start Download**. For mass archiving, click **Queue Visible** followed by **Start Queue**. Queued jobs execute sequentially; completed downloads write atomic `.metadata.json` files and skip on retries.

Metadata Artifacts & Sidecar Layout

Raw Sidecar (`.info.json`)

Direct dump of the yt-dlp internal extractor tree. Contains raw stream URLs, protocol metadata, HTTP headers, full format tables, and player responses directly from YouTube's service.

Normalized Sidecar (`.metadata.json`)

A cleansed, schema-compliant dictionary written upon successful download verification. Includes Title, Channel ID, Upload Date, Views, Likes, Tags, Categories, Codecs, and Resolution.

Diagnostic & Troubleshooting Guide

Observed Symptom	Underlying Root Cause	Resolution Procedure
Only `.json` file on disk	Extraction finished but download crashed or was aborted before stream write.	Verify drive storage limits and network stability. Rerun the download using `Best Available` mode.
Resolution capped at 720p	FFmpeg is unavailable; yt-dlp dropped to single progressive streams.	Install system FFmpeg on PATH or verify `imageio-ffmpeg` exists in the active virtual environment.
Blank / gray thumbnails	Network timeout against `i.ytimg.com` or localized CDN block.	Verify DNS settings. The app will automatically attempt a fallback fetch using `hqdefault.jpg`.
Channel scan pauses	Sequential catalog extraction is processing large video archives.	Do not terminate the application. The live progress HUD monitors active item ingestion.

Automated Test Verification

The project includes pure-function unit tests covering filename sanitization, JSON schema normalization, and format string builders:

```
    # Run unit test suite .venv\Scripts\python.exe -m unittest -v test_app.py # Bytecode syntax chec
```

Legal Framework, Privacy & Compliance

> **Permitted Use Policy:** This software is engineered strictly for personal archiving, metadata indexing, and offline viewing of content the operator has lawful right or authorization to access. Redistribution of copyrighted material without consent is prohibited.

Data Privacy & Local Execution Boundaries

YT Archive operates as an isolated local client. It does not phone home, transmit telemetry, or store user credentials. All metadata extraction queries YouTube public endpoints directly. Location tags shown in metadata indicate author-assigned public geotags, not real-time user or camera coordinates.